

Module 11 — Evals

Evaluating your agent

🎯 Learning goals

- Catch agent regressions with **golden tests**: frozen reference, exact-match tools, judged prose.
- Score retrieval with **recall@k**, **precision@k**, and **MRR**.
- Grade answers with an **LLM-as-judge** (correctness, completeness, relevance).
- Measure **faithfulness**: every claim grounded in retrieved context.
- **Compare** two configs and **gate** a release below a floor.

A live demo is not an eval

One chat	A frozen test set
Sample size: 1	Same questions after every change
Model drift hides behind “it worked on stage”	Cheap checks in CI; token spend on a schedule
No number to compare chunk sizes	A vs B on recall, judge, faithfulness
You find issues when customers tell you about them	Fail the build when a floor is missed

Five layers

- **Agent behaviour** — did it call the right tools? Golden tests vs a frozen reference.
- **Retrieval** — did we fetch the right logs? Recall, precision, MRR.
- **Generation** — is the answer good? LLM-as-judge.
- **Grounding** — did it invent facts? Faithfulness.
- **Release** — is this config better, and good enough? Scorecard + gate.



1. Golden tests

Compare against known good results



What is a golden test?

A **golden test** compares the output of the agent against a known good result. When you make changes to your agent, you can get a numeric score that tells you if you are still good.

run_case → score_case

```
actual = run_case(client, question)      # what the build did
score_case(actual, case, judge)          # vs the frozen reference

# {"id": "warp-status", "score": 0.25, "passed": False,
#  "criteria": {"tools": 0.00, "correctness": 0.25,
#               "completeness": 0.00, "relevance": 0.75}}
```

- Tool calls are structured, so they exact-match: 0 or 1.
- LLM-as-judge scores the prose against the reference on criteria such as correctness, completeness, and relevance.
- An aggregated score gives a hard numerical value — and the criteria say where it went wrong.

Scoring the suite

$$\text{suite} = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{M}\sum_{j=1}^{M} s_{ij} \qquad s_{ij} \in \left[0, 1\right]$$

Two means. The judge scores case i on criterion j — correctness, completeness, relevance — as a continuous value in [0, 1]. Average over the M criteria for a per-case score, then over the N cases for the suite. Because the score is continuous, a paraphrase scores near 1 instead of failing outright.

Equal weights give a plain mean; the part 5 scorecard replaces 1/M with weights that sum to 1. Tool calls stay the exact-match criterion — structured, so 0 or 1.



2. RAG Retrieval metrics

Did we fetch the right documents?

Recall, precision, and MRR

- Start from a **labelled set**: for each query, the document IDs that should come back.
- **Recall@k** is the fraction of gold docs in the top k. Did we miss evidence?
- **Precision@k** is the fraction of the top k that are gold. How much junk hit the prompt?
- **MRR** (mean reciprocal rank) is `1/rank` of the first relevant hit. How far down was the first good doc?

Recall@k and Precision@k

$$\text{Recall@}k = \frac{|\text{top@}k \cap \text{relevant}|}{|\text{relevant}|} \qquad \text{Precision@}k = \frac{|\text{top@}k \cap \text{relevant}|}{k}$$

Precision divides by k even if fewer than k docs were returned. MRR = 1 / rank of the first relevant document (0 if none).

Same query, two rankings

```
gold = ["LOG-002", "LOG-011"]
naive  = ["LOG-007", "LOG-016", "LOG-002"]
rerank = ["LOG-002", "LOG-011", "LOG-007"]

# naive   recall@3=0.50  prec@3=0.33  MRR=0.33
# rerank  recall@3=1.00  prec@3=0.67  MRR=1.00
```

- → Re-run the labelled set after every chunking or index change.
- → Rerank should move gold docs up — the numbers say whether it did.



3. LLM-as-judge

Score meaning, not word overlap

What is LLM-as-judge?

LLM-as-judge asks a strong model to grade a candidate answer against a reference. BLEU and ROUGE count tokens and punish paraphrase (“3500K” vs “3500 Kelvin”). The judge scores **correctness**, **completeness**, and **relevance** from 1–5 and returns JSON. Used to compare prompts and retrieval configs on a frozen set of 20–100 triples.

Zheng et al. (2023). "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena." arXiv:2306.05685

Judge prompt → typed scores

```
class JudgeScores(BaseModel):
    correctness: int = Field(ge=1, le=5)
    completeness: int = Field(ge=1, le=5)
    relevance: int = Field(ge=1, le=5)
    explanation: str

response = client.responses.parse(
    model=MODEL, text_format=JudgeScores, input=[...])
# JudgeScores(correctness=5, completeness=2, relevance=5,
#             explanation="Only one of three causes.")
```

- → Temperature 0. The schema rules out missing or out-of-range scores.
- → Refusal (parsed is None) → scores of 1, do not crash the suite.
- → Treat scores as relative (A vs B), not absolute truth.



4. Faithfulness

Grounded in the sources, not the gold answer

What is faithfulness?

A **faithfulness** check splits the answer into claims and asks whether each one is supported by the **retrieved context** — not by a reference answer. An answer can agree with the gold text and still invent a 9000K core temperature. The score is supported_claims / total_claims. Cousin of citation verification; here you want a **ratio** that sits on a scorecard.

Min et al. (2023). "FActScore: Fine-grained Atomic Evaluation of Factual Precision." arXiv:2305.14251

Claims vs context

```
answer  = "Warp core at 94%. Core temperature is 9000K."
context = "Warp core operating at 94% efficiency. Conduit 7-B fluctuation."

# Claim 1  94%          → supported
# Claim 2  9000K        → unsupported
# faithfulness = 1/2 = 0.5
```

- Zero claims → 1.0 (nothing to hallucinate).
- Check “unsupported” before “supported” — the first word contains the second.



5. Scorecard and gate

Which config wins — and is it good enough?

What is a release gate?

A **scorecard** is retrieval + judge + faithfulness for one pipeline (`rag` vs `rag_rerank`). **Compare** ranks configs on a metric. A **gate** is a floor: if recall drops below 0.6, the build fails even if the demo looked fine. Compare on a delta (“did we get better?”). Gate on a floor (“is this shippable?”).

Reuse this as a release check on a capstone in extras/.

overall + gate

```
judge_norm = (correctness + completeness + relevance) / 15
overall = 0.35*recall + 0.35*judge_norm + 0.30*faithfulness

gate(scorecard, {"avg_recall": 0.6, "overall": 0.55})
# {"passed": False,
#  "failures": [{"metric": "avg_recall", "value": 0.4, "threshold": 0.6}]}
```

- → Equal to the threshold passes; strictly below fails.
- → Precision and MRR stay on the card; overall uses recall + judge + faith.

Summary

Golden tests freeze the outcome

The fixture does not script the agent. Exact-match the tool calls; judge the prose.

Retrieval has numbers

recall@k, precision@k, and MRR, on a small set you labelled properly.

Judge the answer, then check the claims

Correctness, completeness, relevance — then faithfulness against the retrieved context.

Compare on a delta, gate on a floor

“Did we get better?” is a different question from “is this shippable?”

Exercises — five labs

01 Golden tests — frozen reference, score tools + answer, run a suite

02 Retrieval metrics — recall@k, precision@k, MRR on ship logs

03 LLM-as-judge — correctness, completeness, relevance

04 Faithfulness — claims vs retrieved context

05 Eval compare — scorecard, winner, release gate



Module 11 — Complete

Next: productionisation